



UNIVERSIDAD
DON BOSCO

FACULTAD DE INGENIERÍA

Carrera de Ingeniería en Ciencias de la Computación

Programación con Estructura de Datos

Catedrático: Ing. Rafael Torres

EXAMEN PRÁCTICO PARCIAL

Ciclo: 2026 — Ciclo I
Duración: 2 horas (120 minutos)
Nombre: Mario Alexis Pereira Chávez
Eduardo Antonio Peñate Meniivar

Fecha: 22-05-2026
Puntaje: 100 puntos
Carné: PC242674
PM242673

INSTRUCCIONES GENERALES

- El examen es en PAREJA. Cualquier forma de copia anula el examen.
- Puede consultar la documentación oficial de C# (.NET).
- El proyecto debe COMPILAR y EJECUTAR para recibir puntaje. Código que no compila = 0 pts en esa tarea.
- Entregue el proyecto en GitHub
- Crear un video de explicación funcional y del código
- Se evalúa lógica, estructura del código y manejo de casos borde, no solo la salida en consola.

CONTEXTO DEL PROBLEMA

Usted ha sido contratado para desarrollar el Sistema de Navegación del Campus Universitario (SNCU). El campus cuenta con 7 edificios clave interconectados por caminos peatonales. Cada camino tiene una distancia medida en metros.

Los edificios del campus son:

Clave	Edificio	Clave	Edificio
A	Biblioteca Central	E	Gimnasio
B	Cafetería	F	Aulas Generales
C	Laboratorio de Cómputo	G	Estacionamiento
D	Rectoría		

Los caminos peatonales (bidireccionales) con sus distancias en metros son:

Camino	Distancia (m)	Camino	Distancia (m)
A — B	120	C — F	100
A — C	200	D — F	80
B — D	150	E — G	250
B — E	300	F — G	180

TAREAS A DESARROLLAR

TAREA 1 — Construcción del Grafo [20 puntos]

En la clase Grafo.cs, implemente los siguientes métodos:

- `AgregarEdificio(string nombre)`: agrega un nodo al grafo si no existe.
- `AgregarCamino(string origen, string destino, int distancia)`: agrega una arista bidireccional con peso.
- `MostrarGrafo()`: imprime en consola la lista de adyacencia completa de cada edificio.

Salida esperada de `MostrarGrafo()`:

```
=== MAPA DEL CAMPUS ===
Biblioteca Central (A): -> Cafetería (B) [120m] -> Laboratorio (C) [200m] Cafetería
(B): -> Biblioteca (A) [120m] -> Rectoría (D) [150m] -> Gimnasio
(E) [300m]
... (continúa para cada edificio)
```

TAREA 2 — Recorrido BFS [20 puntos]

En la clase Grafo.cs, implemente el método `RecorridoBFS(string inicio)` que:

- Use `Queue<string>` para el recorrido por amplitud (BFS).
- Imprima el orden de visita de los edificios.
- Indique el nivel de distancia (saltos) desde el edificio de inicio.

Llamada en `Program.cs`:

```
grafo.RecorridoBFS("A"); // Iniciar desde Biblioteca Central
```

Salida esperada:

```
=== RECORRIDO BFS desde: Biblioteca Central (A) ===
Nivel 0: Biblioteca Central (A)
Nivel 1: Cafetería (B) | Laboratorio de Cómputo (C)
Nivel 2: Rectoría (D) | Gimnasio (E) | Aulas Generales (F) Nivel
3: Estacionamiento (G)
Total de edificios visitados: 7
```

TAREA 3 — Recorrido DFS [20 puntos]

En la clase Grafo.cs, implemente el método RecorridoDFS(string inicio, string destino) que:

- Use Stack<string> para el recorrido en profundidad (DFS).
- Busque si existe un camino desde 'inicio' hasta 'destino'.
- Imprima el camino encontrado (secuencia de edificios recorridos).
- Indique si el camino fue encontrado o no.

Llamada en Program.cs:

```
grafo.RecorridoDFS("A", "G"); // ¿Hay camino de Biblioteca a Estacionamiento?
```

Salida esperada:

```
=== RECORRIDO DFS: Biblioteca Central (A) --> Estacionamiento (G) ===  
Visitando: Biblioteca Central (A)  
Visitando: Cafetería (B)  
... (camino encontrado según el recorrido)  
✓ Camino encontrado: A -> B -> E -> G
```

TAREA 4 — Tabla Hash de Visitas [20 puntos]

En la clase TablaHash.cs, implemente un registro de visitas usando Dictionary<string, int>. Los métodos requeridos son:

- RegistrarVisita(string edificio): incrementa el contador de visitas del edificio.
- ObtenerConteo(string edificio): retorna cuántas veces fue visitado.
- MostrarEstadisticas(): imprime todos los edificios con su conteo de visitas, ordenados de mayor a menor.
- EdificioMasVisitado(): retorna el nombre del edificio con más visitas.

En Program.cs, después de ejecutar BFS y DFS, registre cada visita y llame a MostrarEstadisticas():

Salida esperada:

```
=== ESTADÍSTICAS DE VISITAS ===  
Cafetería (B) : ██████████ 4  
visitas Biblioteca Central (A) : ████████ 3  
visitas Estacionamiento (G) : ██████ 2  
visitas  
... (continúa)  
Edificio más visitado: Cafetería (B) con 4 visitas
```

TAREA 5 — Min-Heap de Rutas [20 puntos]

En la clase MinHeap.cs, implemente un heap mínimo que almacene rutas como tuplas (edificio, distancia). Los métodos requeridos son:

- Insertar(string edificio, int distancia): inserta una ruta en el heap.
- ExtraerMinimo(): extrae y retorna la ruta de menor distancia.
- EstaVacio(): indica si el heap está vacío.
- MostrarRutasOrdenadas(): extrae todos los elementos e imprime las rutas de menor a mayor distancia.

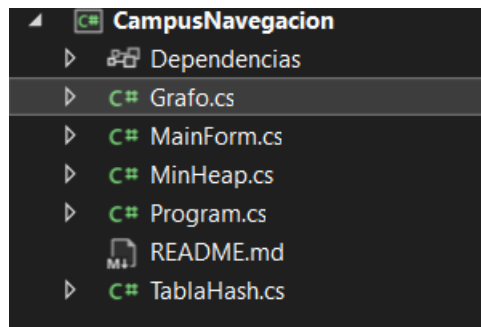
En Program.cs, inserte **TODAS** las rutas directas desde la Biblioteca Central (A) y muéstrelas ordenadas:

Salida esperada:

```
=== RUTAS DESDE Biblioteca Central (A) — Ordenadas por distancia ===  
1° Cafetería (B) 120 metros  
2° Laboratorio de Cómputo (C) 200 metros  
Heap vacío — todas las rutas fueron procesadas.
```

ESTRUCTURA DEL PROYECTO

Su proyecto debe seguir exactamente esta estructura de archivos:



RÚBRICA DE CALIFICACIÓN

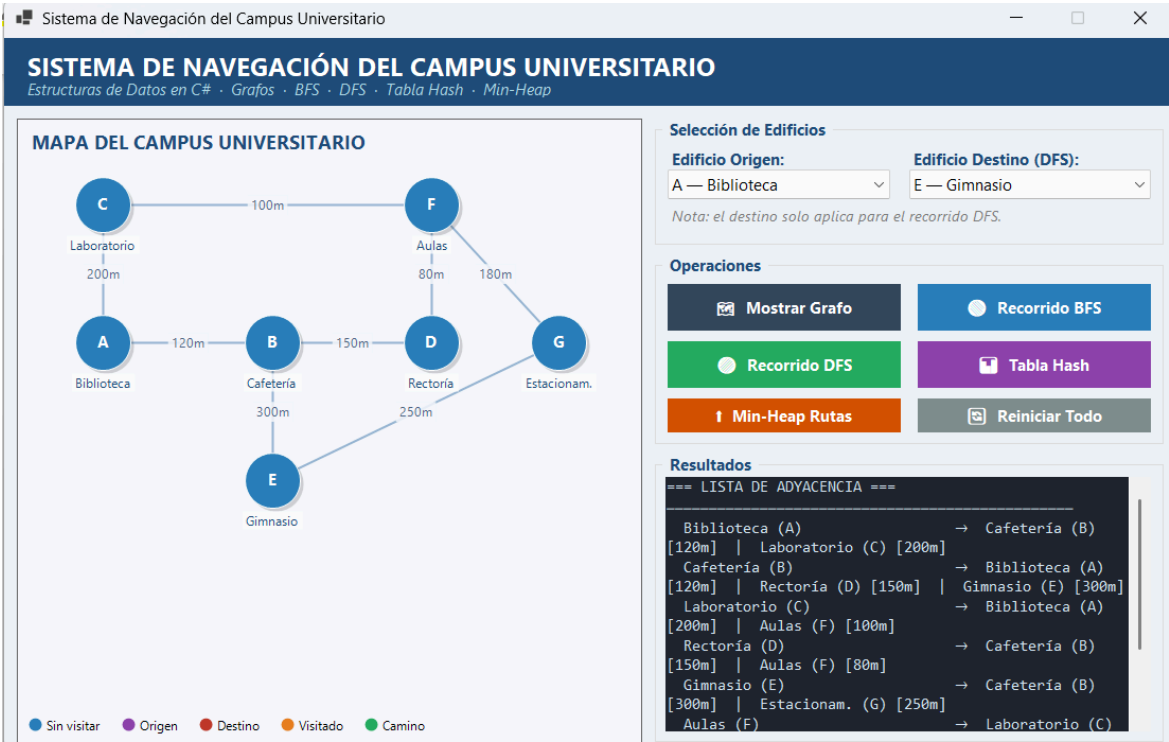
Tarea	Criterio evaluado	Pts.	Subtotal
Tarea 1 — Grafo	Estructura de datos correcta + aristas bidireccionales + impresión	20	/20
Tarea 2 — BFS	Uso correcto de Queue + orden de visita + niveles correctos	20	/20
Tarea 3 — DFS	Uso correcto de Stack + detección de camino + camino impreso	20	/20
Tarea 4 — Tabla Hash	Dictionary correcto + estadísticas ordenadas + edificio más visitado	20	/20
Tarea 5 — Min-Heap	Heap manual funcional + inserción + extracción + orden correcto	20	/20
TOTAL		100	/100

⚠ IMPORTANTE

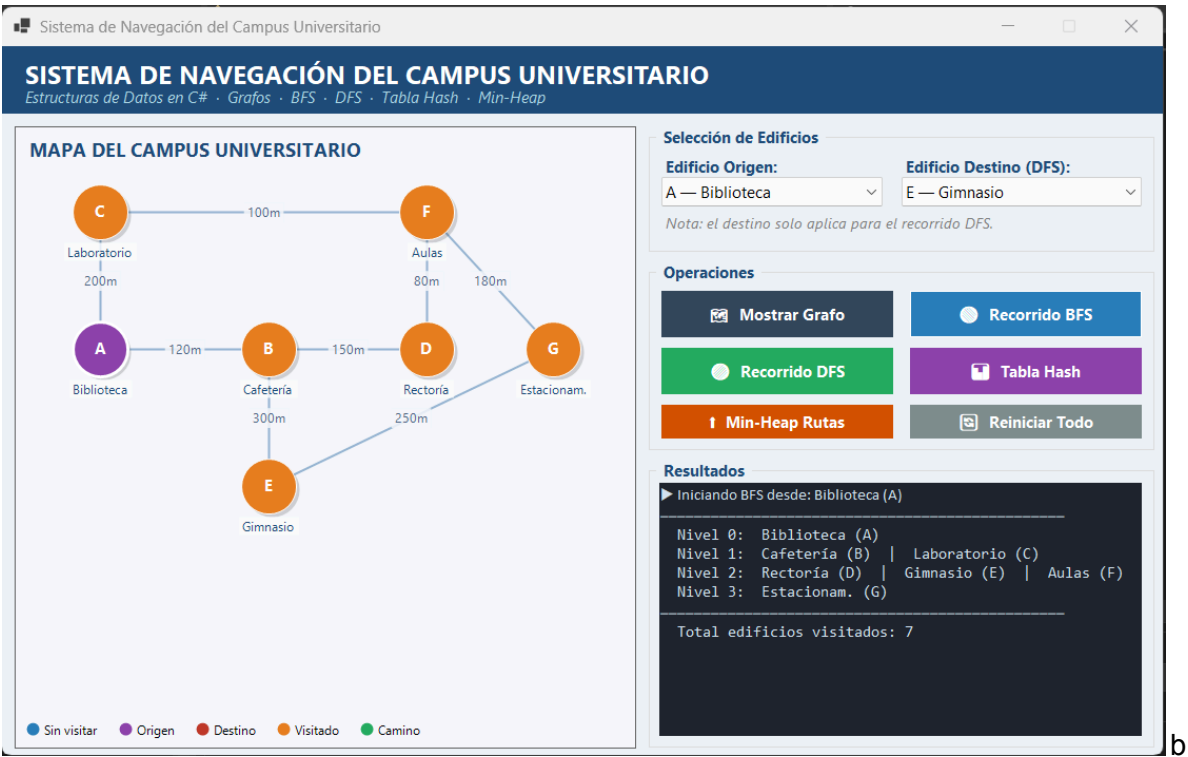
El proyecto debe compilar sin errores para recibir puntaje. Se descuentan 5 pts por cada advertencia de compilación no resuelta. Código comentado o métodos vacíos sin implementar = 0 pts en esa sub-tarea.

Imágenes de Funcionamiento

Mostrar Grafo



Recorrido BFS



Recorrido DFS

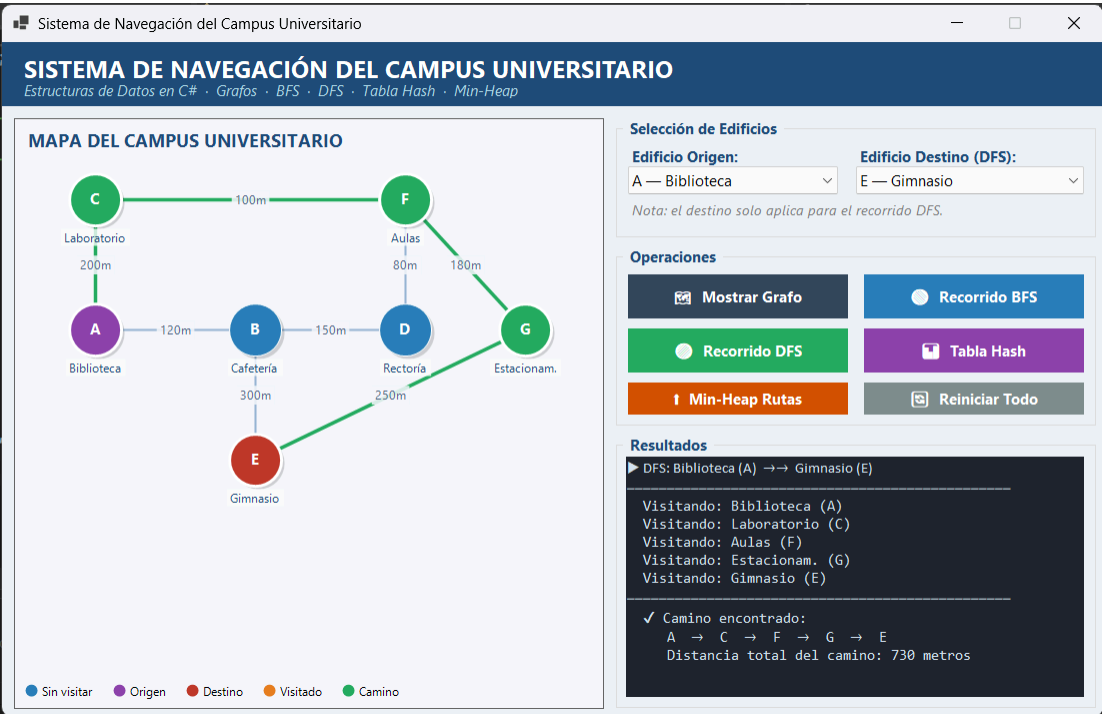
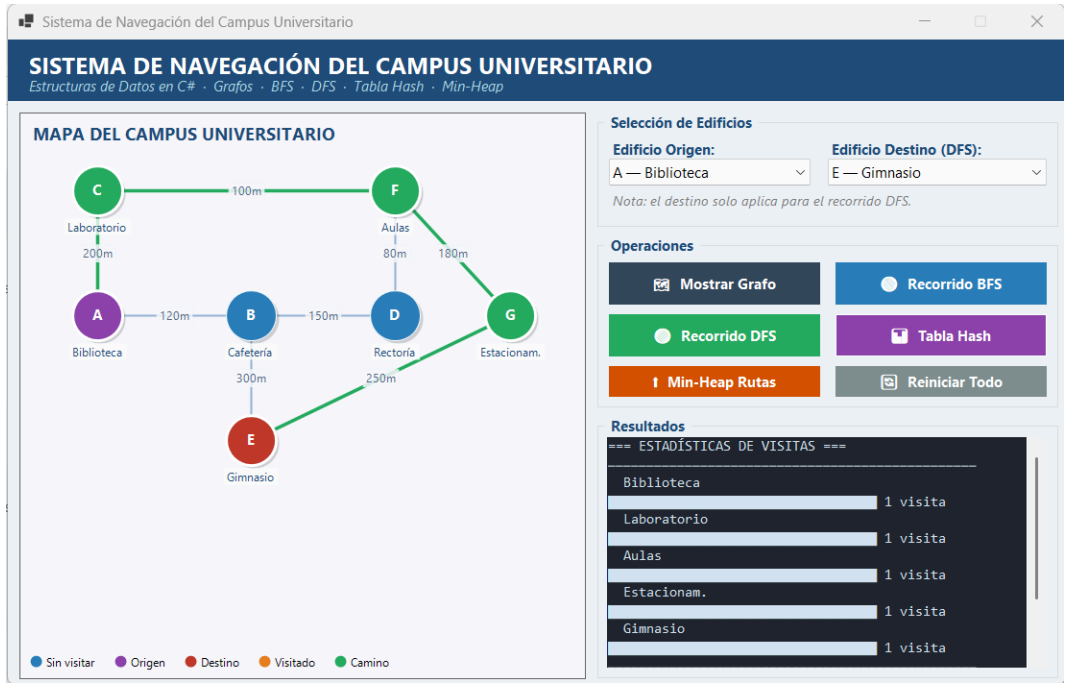


Tabla HASH



Mini-Heap Rutas



¡Mucho éxito! — Recuerde: un programa que hace algo correcto vale más que uno que no compila.